# Language and Command Reference

Giolit Trading Risk Verifier 1.0.0 — the GTSL language, every option, the library

## Contents


1. A first run . . . 1
2. How the verifier sees trading . . . 2
3. GTSL, section by section . . . 3
4. Expressions, units and names . . . 4
5. Rules . . . 5
6. Risk controls . . . 6
7. Constraints . . . 6
8. Reading the results . . . 6
9. Risk controls for any strategy . . . 7
10. Risk headroom . . . 7
11. Certificates and reports . . . 7
12. Conformance replay of FIX logs . . . 8
    Verification in CI (a release gate) . . . 8
13. FIXatdl and FIX Orchestra . . . 9
14. Optimization . . . 9
15. Several venues and concurrent trading . . . 9
16. Real numbers (binary64) . . . 10
17. Profit and loss: what can be guaranteed . . . 11
18. Several symbols, legs, signals and history . . . 11
19. The strategy pattern library . . . 13
20. Verifying your C++ risk checks from their source . . . 13
    Importing the strategy itself . . . 14
21. What a result means . . . 15
22. The library API . . . 15
23. Command reference . . . 16


## 1. A first run

The problem statement's example: a risk check looks at the position only, so two buy orders of 100 from a position of 900 both pass and both fill.

```
spec PositionLimit;

units { price = tick(0.05 INR); money = paise; }
ranges { price 20000 .. 40000; order_qty 1 .. 500; }

strategy Accumulate {
    parameters {
        initial_position = 900;
        max_position = 1000;
    }
    rules {
        on market_update {
            buy(100, limit = market.bid);
        }
    }
}

risk_controls {
    position_cap: reject when position + order.qty > max_position;
}

constraints {
    position_limit: always position <= max_position;
}
```

```
$ trade-verify examples/01_position_limit.gts
  VIOLATED    position_limit  always position <= max_position   (line 25)
              violating run, 6 events (counterexample (simulation and guided search) [slot model]):
             0  initial state
             1  MarketUpdate bid=31124 ask=28373 last=36065 volume=1430
                  -> NewOrder O1 BUY 100 @ 31124
             2  Ack O1
             3  MarketUpdate bid=20003 ...
                  -> NewOrder O2 BUY 100 @ 20003
             4  Ack O2
             5  Fill O2 100 @ 20000
             6  Fill O1 100 @ 20000
              the constraint fails in the state after the last event
Result: CONSTRAINT VIOLATIONS FOUND (1 violated)
```

Change the control to `reject when position + open_qty(buy) + order.qty > max_position` (example 02) and the result is `PROVEN ... [aggregate model: any number of orders]`.

## 2. How the verifier sees trading

A specification describes a **transition system**: a state (time, market, position, cash, orders, the strategy's own state) and the events that change it. One step is one event:

| Event | From | Effect |
|---|---|---|
| `market_update` | the market | new bid, ask, last (any values within the price range and the assumptions); the volume grows |
| `clock` | time | nothing but time passing (time advances before every event, by any amount within the session) |
| `ack`, `reject` | the venue | a pending new order is acknowledged or rejected |
| `fill` | the venue | a live order fills partly or completely (quantity 1 to its leaves; a buy at or below its limit) |

| Event | From | Effect |
|---|---|---|
| `cancel_ack`, `cancel_reject` | the venue | a pending cancel is confirmed or refused |
| `exchange_cancel` | the venue | (protocol option `exchange_cancels`) the venue cancels a live order on its own |
| `kill_switch` | the operator | the kill switch is switched on (once; it stays on) |
| `session_close` | time | the last event of a session (with `session HH:MM .. HH:MM`) |

After the event, the strategy's **rules** for that event run; every order they send passes the **risk controls**; the **assumptions** are imposed; then the **constraints** are checked on the new state. The verifier explores every sequence of events and every value the market and the venue can choose.

**Orders** follow the FIX order lifecycle:

```
NewOrder         Ack               PartialFill*        Fill
————————> PendingNew ——> New ————————————————> PartFilled ——> Filled
             │ Reject     │ CancelRequest       │ CancelRequest
             v            v                     v
          Rejected    PendingCancel —— CancelAck —> Cancelled
                           │  (fills can still arrive before the CancelAck)
                           └—— CancelReject —> back
```

**Two models of the same specification** are verified side by side:

- the **slot model** tracks up to K live orders (default 4, `--slots`) with their exact lifecycle. Every violation it reports is a real run;
- the **aggregate model** tracks the total open quantity per side (and one watched order for per-order constraints). Its proofs hold **for any number of orders**.

## 3. GTSL, section by section

A file starts with `spec Name;` and has these sections in any order (each optional except where noted).

```
spec Name;

units { time = 1ms; price = tick(0.05 INR); money = paise; }
ranges { price 20000 .. 40000; order_qty 1 .. 500; volume_step 0 .. 5000; time_step 0 .. 1s; }
protocol fix44.limit_orders { late_fills; exchange_cancels; }      // or: protocol from orchestra "venue.
    xml";
market symbolic { symbol INFY; reference_price = 1500.00 INR; }
venues { NSE; BSE; }                                               // optional: several venues (section
    15)
assumptions { session 09:15 .. 15:30; market.bid <= market.ask; initial position <= 1000; on fill: fill.qty
     <= 500; }
mode controls_only;
strategy Name { parameters { ... } state { ... } rules { ... } }
risk_controls { name: reject when CONDITION; ... }                 // risk_controls async { ... }: section
     15
constraints { name: CONSTRAINT; ... }
objective { minimize EXPRESSION; }
```

| Section | Contents |
|---|---|
| `units` | `time` resolution (default `1ms`); the price `tick` (default 0.01) and its currency; the `money` unit (`paise`, `cents`, or an amount such as `0.0001 USD`). A tick must be a whole number of money units. |
| `ranges` | the values the market and the venue can choose: `price LOW .. HIGH` (in ticks, or amounts such as `1500.00 INR`), `order_qty LOW .. HIGH`, `volume_step 0 .. N` (how much market volume one update adds), `time_step 0 .. D` (how far time moves per event; default: up to the whole session). |
| `protocol` | `fix44.limit_orders` (built in) with options `late_fills` (the venue may fill an order after it acknowledged the cancel), `exchange_cancels` (the venue may cancel live orders on its own, e.g. expiry) and `fill_reports` (a fill arrives as two messages in either order, with other events between them: the trade, which books position and cash and runs the rules `on fill`, and the order's status report, which runs the rules `on fill_report`; the order's leaves, state and open totals change at whichever comes first. Brokers such as CTP send these separately: OnRtnTrade, OnRtnOrder). |
| `market` | `symbol NAME;` and market parameters (`name = value;`). |
| `venues` | the venues the symbol trades on, the first being the primary (section 15). Without it: one venue. |
| `assumptions` | `session OPEN .. CLOSE;` (time starts at OPEN; `session_close` ends the run at CLOSE); `initial COND;` (holds in the initial state); `COND;` (holds after every event); `on EVENT [where COND]: COND;` (holds at that event, e.g. about `fill.price`). |
| `mode controls_only;` | any strategy instead of the rules (section 9). |
| `strategy` | `parameters` (constants; `name = any;` for an arbitrary initial value), `parameters from fixatdl ...` (section 13), `state` (`name: TYPE = INITIAL;` with TYPE `qty`, `price`, `money`, `int`, `bool`, `time`, `duration`, or `real [price\|qty\|money] in [LOW, HIGH]` (section 16)), `rules` (section 5). `name = any real in [LOW, HIGH];` is any binary64 value of the range. |
| `risk_controls` | `reject when` conditions applied to every outgoing order (section 6); `risk_controls async` for checks that see the last published state (section 15). |
| `constraints` | what must hold (section 7). |
| `objective` | `minimize` or `maximize` an expression (section 14). |

Parameters with special meaning: `initial_position`, `initial_cash` (the start), `arrival_price` (for `implementation_shortfall`).

## 4. Expressions, units and names

**Literals.** `1_000_000`, `12.5`, `10%`, `09:30`, `09:30:15`, `50ms`, `2s`, `5min`, `1h`, `20 ticks`, `1500.00 INR` (an amount: a price where a price is expected, money where money is expected), `true`, `false`, `buy`, `sell`.

**Operators.** `+ - * / %`, comparisons `== != < <= > >=`, `and or not` (also `&& || !`), `=>` (implies), `x in [a, b]` / `[a, b)`

/ `(a, b]`, `x within 10% of y`, `x within 20 ticks of y`.

**Exact arithmetic.** Everything is an integer in its unit: prices in ticks, money in money units, quantities in shares, time in time units. Division by a constant truncates toward zero; `x * 10%` is `x * 10 / 100`, truncated. A ratio such as `participation < 10%` or `a / b < c` is compared exactly by cross-multiplication (a zero divisor counts as a ratio of 0). A division by a variable used as a value is truncated, and 0 when the divisor is 0.

**Units are checked.** Adding a price to a quantity, or comparing money with time, is an error. `quantity × price` is money; `time - time` is a duration; a plain number adapts to the other side.

**Names.**

| Name | Meaning |
|---|---|
| `position`, `bought`, `sold`, `traded` | net position; shares bought, sold, both (since the start) |
| `cash` | money received minus money paid |
| `open_qty(buy)`, `open_qty(sell)`, `open_qty()` | the leaves of the live orders of a side (or both) |
| `open_orders` | the number of live orders |
| `reported_open_orders` | the live orders as the strategy's code knows them: with `fill_reports`, an order a trade completed counts until its status report; else `open_orders` |
| `participation` | `traded / market.volume` (a ratio: compare it with a percentage) |
| `implementation_shortfall` | `-cash - (position - initial_position) × arrival_price` (needs `arrival_price`) |
| `market.bid`, `.ask`, `.last`, `.volume`, `.spread` | the market |
| `time`, `kill_switch` | the clock; the operator's switch |
| `order.side`, `.qty`, `.price`, `.cum_qty`, `.leaves`, `.is_market`, `.last_qty`, `.last_price`, `.notional` | the order of an order event (also through the handler's or constraint's order variable: `o.qty`) |
| `fill.qty`, `fill.price` | at fill events |
| `notional(order)` | `qty × price` (a market order is valued at the opposite quote) |
| `prev(e)` | `e` at the start of the current step |
| `cancelled(o)`, `filled(o)`, `rejected(o)`, `pending_cancel(o)`, `pending_new(o)`, `live(o)`, `acked(o)`, `done(o)` | order states |
| `min(a, b, ...)`, `max(...)`, `abs(x)` | |

## 5. Rules

```
rules {
    on market_update when time in [start, end) and not kill_switch {
        let target = quantity * (time - start) / (end - start);
        let gap = target - bought - open_qty(buy);
        if gap > 0 and participation < max_participation {
            buy(min(child_max, gap), limit = market.bid);
        }
    }
    on fill(o) { executed += o.last_qty; }
    on kill_switch { cancel_all(); }
}
```

- Events: `market_update`, `clock`, `kill_switch`, `session_close`, `start` (once, in the initial state), and the order events `ack(o)`, `reject(o)`, `fill(o)`, `fill_report(o)`, `cancel_ack(o)`, `cancel_reject(o)`, `exchange_cancel(o)`. `fill_report` is the order's status report of a fill: with the protocol option `fill_reports` a message of its own; without it, its rules run in the fill's step, before the rules `on fill`.

- Statements: `let name = e;`, `x = e;`, `x += e;`, `x -= e;` (state variables), `if c { } else { }`, and the actions `buy(qty)`, `buy(qty, limit = price)`, `sell(...)`, `cancel(o)`, `cancel_all()`.
- A `buy` or `sell` of 0 shares or less sends nothing. Every order passes the risk controls first.
- `assert NAME: COND;` states a fact at that point of a rule: it is checked there (the check `assert:NAME`, one for every place the name stands) and then assumed, which gives the provers an invariant at that point. Like declared ranges, it is never taken on trust: the other proofs count only when every assertion is proven.

## 6. Risk controls

```
risk_controls {
    clip:      reject when order.qty > 500;
    long_cap:  reject when position + open_qty(buy) + order.qty > 1000;
    notional:  reject when notional(order) > 50_00_000.00 INR;
    kill:      reject when kill_switch;
}
```

Each control sees the candidate `order` and the current state. An order any control rejects is not sent (it counts in `rejected_by_controls` and is the event `control_reject`).

## 7. Constraints

| Form | Meaning |
|---|---|
| `always COND` | after every event |
| `always on EVENT [where COND]: COND` | whenever the event happens (state after the event's own effect) |
| `after TRIGGER: never EVENT [where COND]` | once TRIGGER has held, the event never happens again |
| `after TRIGGER: always COND` | once TRIGGER has held, COND holds after every event |
| `never EVENT [where COND]` | the event never happens |
| `for each order o: ...` | any of the forms above for every order (`o` in the conditions) |
| `at end: COND` | at the session's close |
| `feasible COND`, `feasible at end: COND` | a **target**: some run reaches it (Q3) |

Events for constraints: the rule events, plus `new_order`, `cancel_request` and `control_reject`.

```
constraints {
    position_limit: always position <= 1000;
    order_size:     always on new_order: order.qty <= 500;
    kill:           after kill_switch: never new_order;
    cancel_final:   for each order o: after cancelled(o): never fill(o);
    accounting:     always position == initial_position + bought - sold;
    completes:      feasible at end: bought == quantity;
}
```

## 8. Reading the results

| Verdict | Meaning |
|---|---|
| `PROVEN` | holds on every run, for any number of orders |

| Verdict | Meaning |
|---|---|
| `PROVEN*` | holds on every run with at most K live orders at a time (`--slots`); not proved beyond |
| `VIOLATED` | a run breaks it; the run is shown (shortened) |
| `FEASIBLE` / `INFEASIBLE` | a target is reached by a run (shown) / proved unreachable |
| `UNDECIDED` | neither within the time budget (`--timeout`); the cause is shown |

Each verdict names its method: *inductive lemmas* or *invariant* (proofs), *simulation and guided search* (runs), and the order model that decided it. A run lists one event per line, the strategy's actions (`->`), the state variables that changed, and the orders.

The **sanity checks** run with every verification: that the assumptions allow a run at all (otherwise everything is proven vacuously), that every rule can fire, every action can execute, every risk control can reject an order, every trigger can occur. `--verbose` lists them; failures are always shown as warnings.

## 9. Risk controls for any strategy

`mode controls_only;` (or `--controls-only`) replaces the rules by **any strategy**: at every step it may send any buy or sell order of any size within `order_qty`, at market or with any limit within the price range, or cancel any order. A PROVEN result then certifies the risk controls themselves – for every algorithm the desk may ever run. See `06_controls_only.gts` (all proven) and `07_controls_short_gap.gts` (the sell side is not capped: VIOLATED).

## 10. Risk headroom

```
$ trade-verify --headroom examples/06_controls_only.gts
  PROVEN      long_limit   always position <= max_long
              headroom: position never exceeds 1000 (reached by a run); limit max_long = 1000; 0% of the
    limit unused  *....
```

For every proven `always x <= L` (or `>=`, also `always on EVENT: x <= L`), the verifier tightens L step by step and proves again: the result is the worst value x reaches on any run, whether a run reaches it, and how much of the limit is never used. The **Giolit Risk Resilience Score** rates it: 5 stars from 50% unused, 4 from 25%, 3 from 10%, 2 above 0, 1 when a run reaches the limit. Controls designed to stop exactly at a limit score 1 star – the limit is fully usable; a strategy proven to stay far below its limit scores higher.

## 11. Certificates and reports

```
trade-verify examples/06_controls_only.gts --report=bundle --issued-to="Example Broking Pvt Ltd"
```

| File | |
|---|---|
| `certificate.pdf` / `violation-report.pdf` / `verification-report.pdf` | the document (every constraint proven / a violation / otherwise) |
| `certificate.json` (+ `.sig`) | the statement: verdicts, the specification's SHA-256, every evidence file's SHA-256; signed by the licensee (full edition) |
| `proof.json` | each proof as relations over the trading state (e.g. `open_qty(buy) + position <= 1000`) and as the engine stated it |

| File | |
|---|---|
| `runs/<constraint>.txt`, `.json`, `.fix`, `.csv` | every violating or witness run: text, JSON, FIX 4.4 messages, the market path |
| `verify_certificate.py`, `signer-credential.*`, `giolit-master-key.txt` | the chain of trust (full edition) |
| `reproduce.sh`, `results.json`, `README.txt` | |

An auditor checks a bundle with `python3 verify_certificate.py bundle`: the signatures, the specification file and every evidence file. A proof in the certificate is an **inductive invariant**: it holds initially, every event preserves it, and it excludes every violation.

## 12. Conformance replay of FIX logs

A proof covers the specification. Conformance replay checks that the specification describes production:

```
$ trade-verify examples/06_controls_only.gts --conform=examples/logs/06_late_fill_from_venue.fix
  IMPOSSIBLE message 7 (line 10): a fill needs a live acknowledged order (New, PartFilled or PendingCancel)
  CONTROLS   message 8 (line 11): the risk controls of the specification reject this order, but it was sent
      to the venue
Result: THE LOG DOES NOT CONFORM TO THE SPECIFICATION
```

The log is FIX tag=value (fields separated by SOH or `|`; lines starting with `#` are comments). Messages used: `35=W/X` market data (269/270), `35=D` new orders (11, 54, 38, 40, 44), `35=F` cancel requests (11, 41), `35=8` execution reports (11/41, 150: 0 ack, 8 reject, F/1/2 fill with 32/31, 4 cancelled), `35=9` cancel rejects; times from 60 or 52. Events that are not FIX messages are lines `#! kill_switch`, `#! session_close` or `#! clock`, optionally followed by `|60=<time>|`; the runs that `--traces` and `--report` write use them, so every run replays as it was found.

| Kind | Meaning |
|---|---|
| `IMPOSSIBLE` | the specification says this message cannot happen (a venue behaviour or market value it excludes) |
| `UNEXPECTED` | the strategy sent an order or cancel its rules would not send at this point |
| `MISSING` | the rules would send an order the log does not show |
| `CONTROLS` | (controls-only) an order the specification's risk controls reject reached the venue |
| `VIOLATION` | the observed run violates a constraint |
| `UNMAPPED` | a message about an order the replay does not know (after an earlier deviation) |

Run it nightly on drop copies: it turns the assumptions into something monitored. With `--follow` it watches a log that is still being written (a live drop copy) and prints each new deviation as it appears:

```
$ trade-verify spec.gts --conform=/var/log/dropcopy/today.fix --follow=5
```

With `fill_reports` a FIX fill (35=8, 150=F/1/2) is the trade followed at once by its status report.

### Verification in CI (a release gate)

```
$ trade-verify strategy.gts --timeout=120 --json=results.json --junit=results.xml --baseline=release-1.4.
    json
```

- `--junit` writes JUnit XML, which GitHub, GitLab and Jenkins show per test: one test case per constraint, a violation as a failure with its run, UNDECIDED as skipped.
- `--baseline` compares with the `--json` of the last release: a constraint it proved that is no longer proven is a `REGRESSION` (exit code 5 when nothing is violated); `FIXED`, `NEW` and `REMOVED` constraints are listed too.
- Exit codes: 0 all proven, 1 violated, 2 undecided, 3 input error, 4 license, 5 regression against the baseline. A strict gate fails on anything but 0; a lenient one on 1 and 5.

`share/giolit/ci/` (in the installation) has a GitHub Actions workflow and a GitLab CI job that run every specification of a repository and keep the results of the default branch as the next baseline.

## 13. FIXatdl and FIX Orchestra

**FIXatdl.** Verify a strategy for **every parameter value a client can enter** in the OMS:

```
strategy VWAP {
    parameters { quantity = 1000; }
    parameters from fixatdl "imports/algos.fixatdl.xml" strategy "VWAP" {
        start = StartTime;  end = EndTime;  max_participation = ParticipationRate;  child_max = MaxChildQty
    ;
    }
    ...
}
```

Each mapped parameter becomes an arbitrary value within the FIXatdl `minValue`/`maxValue` (`constValue`: fixed). Types: `Qty_t`, `Int_t`, `Price_t`, `Amt_t`, `Percentage_t`, `LocalMktTime_t`/`UTCTimestamp_t`, `Boolean_t`; other types are reported as not modelled. A parameter the `parameters` section sets keeps that value.

**FIX Orchestra.** `protocol from orchestra "venue.xml";` reads the venue's order state machine (`fsm`/`state`/`transition`), switches on `late_fills` when it fills cancelled orders and `exchange_cancels` when it cancels or expires live orders, and lists every transition the model does not express.

## 14. Optimization

```
objective { minimize implementation_shortfall; }
```

```
$ trade-verify --optimize examples/11_execution_cost.gts
  objective: minimize implementation_shortfall
    best (lowest): a run reaches -3549355
    worst (highest): a run reaches 4999985; proved: no run goes above 5000005
```

Over all behaviours, the market is adversarial: the verifier reports the **best** and the **worst** value of the objective (at the session's close; without a session, at any step), each as a value a concrete run reaches and a bound proved for every run. When the two meet, it is the exact optimum. Stochastic objectives (expected return, market impact) need a probabilistic market model and are not part of this version.

## 15. Several venues and concurrent trading

A strategy that trades one symbol on several venues declares them; the first is the primary venue.

```
venues { NSE; BSE; }
assumptions {
    market.bid < market.ask;                              // every venue's book
    market[NSE].ask within 20 ticks of market[BSE].ask;
    on market_update: market.bid within 50 ticks of prev(market.bid);   // the venue that updates
}
strategy Router {
    rules {
        on market_update {                                // market.* here: the venue that updated
            buy(10, limit = market.bid);                  // sent to that venue
            buy(10, limit = market[BSE].ask, venue = BSE);
        }
        on kill_switch { cancel_all(); }                  // every venue; cancel_all(NSE): one venue
    }
```

```
}
risk_controls {
    venue_cap:  reject when open_qty(buy, order.venue) + order.qty > 100;   // per venue
    global_cap: reject when open_qty(buy) + order.qty > 150;                // all venues
}
```

| Name | Meaning |
|---|---|
| `market[V].bid`, `.ask`, `.last`, `.spread` | the book of venue V |
| `market.bid`, … | the venue of the context: the venue that updated (`on market_update`), the order's venue (order events, risk controls, `on new_order` constraints), the primary venue otherwise; in an assumption or an `always` constraint without an order, **every** venue |
| `order.venue`, `fill.venue`, `event.venue` | a venue (compare with `==`, `!=`) |
| `open_qty(buy, V)`, `open_orders(V)` | the open quantity and the live orders at venue V; `open_qty(buy)`, `open_orders`, `position` stay global |
| `buy(q, limit = p, venue = V)` | an order to venue V (default: the venue of the context) |

Every venue has its own market data, order books and messages: the messages of different venues interleave in any order, which is where most multi-venue failures come from – per-venue limits that add up past a global one, a router that sweeps several venues for the same quantity (both fill: the parent overfills), a kill switch that reaches one venue. `examples/14_smart_order_router.gts` shows two of them; `15_...fixed.gts` the fixes. Traces and FIX logs name the venue: `ExDestination (100)` on orders, `LastMkt (30)` on executions, `SecurityExchange (207)` on market data, and `--conform` replays them.

**Asynchronous risk checks.** In many engines the risk check runs on a thread of its own and reads the shared position and open quantities as last published by the order gateway. `risk_controls async { ... }` models exactly that: the controls see `position`, `open_qty`, `open_orders`, `bought`, `sold` and `cash` as last published, and publishing is an event of its own that can happen at any time. Two orders can then pass a check against the same stale position (check-then-act). The default, synchronous controls, see the current state.

**Scale.** Proofs in the aggregate model cover any number of orders on every venue. Measured on a laptop (one run at a time, 8 GB cap): a global limit across 16 venues is proved in 21 s (1.9 GB); a violation that needs 49 live orders on 16 venues is found in 44 s (4.5 GB). Two and four venues take under a second and about 2 s.

## 16. Real numbers (binary64)

Strategies written with `double` in production can be written the same way:

```
strategy MarketMaker {
    parameters { gamma = 0.002; kappa = any real in [0.5, 3.0]; lambda = 0.94; }
    state {
        sigma2:   real in [0.0, 2600.0] = 4.0;              // the declared range is checked (range:sigma2)
        last_mid: real price in [0.0, 50000.0] = 0.0;
    }
    rules {
        on market_update {
            let mid = real(market.bid + market.ask) / 2.0;
            let d = min(max(mid - last_mid, -50.0), 50.0);
            sigma2 = lambda * sigma2 + (1.0 - lambda) * d * d;
            buy(10, limit = floor(mid - real(position) * gamma * sigma2));
        }
    }
}
```

- A `real` is an IEEE-754 binary64 value, computed exactly as C/C++ `double` (round to nearest, no fused multiply-add). Counterexamples show the exact values. `real(x)` turns an integer into a plain number

in model units (ticks, shares); `floor`, `ceil`, `round`, `trunc` turn a real back into an integer (a price where a price is expected).
- Functions: `exp`, `ln`/`log`, `sqrt`, `min`, `max`, `abs`. Literals may have exponents (`1e-4`).
- Every `real` state variable declares a range, checked after every update (`range:NAME`); the other proofs count only when every range check is proven. A value that may become NaN or infinite gets a check `float@LINE`.
- Proofs work on sound linear relations around the real computations. `floor`, `ceil` and `trunc` of a real state variable are related to it exactly, so a property at the boundary of the declared range (`floor(ema ) >= 100` with `ema` in `[100.0, ...]`) is proved. A bound that only a fixpoint of the update gives exactly (an average that never falls below the lowest market price) may still need a little slack.
- `examples/12_avellaneda_stoikov.gts` is the Avellaneda-Stoikov market maker (a bug: skewed quotes cross the market); `13_..._fixed.gts` clamps the quotes and proves every constraint.

## 17. Profit and loss: what can be guaranteed

Against every market the specification allows, no strategy that holds inventory can be proved profitable: the market can always move against the position. What can be proved, for every run:

```
constraints {
    edge_locked_in:  always cash + position * bid_level >= sold * (ask_level - bid_level);   // every share
      sold earned the spread
    worst_case_loss: always cash + position * 20000 ticks >= -max_loss;                      // the loss if
      the price collapses
    never_loses:     always cash + position * market.bid >= 0;                                // VIOLATED:
     marked to market
}
```

- A buy fills at or below its limit and a sell at or above it. When every buy of the rules carries a constant limit (and every sell), the verifier uses those limits as the prices fills can have, so the cash a strategy locks in is proved exactly: `edge_locked_in` holds for any market, any fill sizes, any number of orders.
- With an inventory limit, the worst-case loss is bounded: `worst_case_loss` values the inventory at the lowest price of the range.
- Marked to the current market, any strategy can be under water: `never_loses` gets a three-event run (buy, the market falls). Expected profit needs a probabilistic market model and is not part of this version.

`examples/16_market_maker_pnl.gts` has all three. A strategy whose limits follow the market (`limit = market.bid`) gets no such proof, since its fills can be at any price of the range; its losses are still found as exact runs.

## 18. Several symbols, legs, signals and history

Pairs, index arbitrage, hedging and spreads trade several symbols. Each symbol has its own book (on every venue), its own position and fills; cash is shared.

```
symbols {
    HDFCBANK  price 30000 .. 34000 beta 1.0;     // its own price range (ticks) and its beta
    ICICIBANK price 18000 .. 22000 beta 1.2;
}
signals { zscore: int in [-400, 400]; }         // a model's output: any value of the range at every market
      update
strategy Pairs {
    parameters { clip = 100; hedge = 150; entry = 200; }
    rules {
        on market_update when zscore > entry and open_orders(HDFCBANK) == 0
                          and market[HDFCBANK].last >= history(market[HDFCBANK].last, 3) {
            legs {                                  // both orders pass the risk controls, or neither is
     sent
                sell(clip, limit = market[HDFCBANK].bid, symbol = HDFCBANK);
                buy(hedge, limit = market[ICICIBANK].ask, symbol = ICICIBANK);
            }
        }
    }
}
risk_controls {
```

```
    gross: reject when gross_exposure + notional(order) > 4000000 INR;
}
constraints {
    pair_imbalance: always abs(3 * position[HDFCBANK] + 2 * position[ICICIBANK]) <= 300;
    beta_hedged:    always beta_exposure <= 2000000 INR;
}
```

| Name | Meaning |
|---|---|
| `market[S].bid`, `market[S, V].bid`, `.mid` | the book of symbol S (at venue V); `market.bid`: the symbol (and venue) of the context |
| `position[S]`, `bought[S]`, `sold[S]`, `traded[S]`, `open_orders[S]` | per symbol; without `[S]`: the symbol of the context (the order's, the event's) |
| `open_qty(buy, S)`, `open_qty(buy, S, V)`, `open_orders(S)` | the open quantity / live orders of a symbol (at a venue) |
| `order.symbol`, `fill.symbol`, `event.symbol` | a symbol (compare with `==`, `!=`) |
| `buy(q, limit = p, symbol = S, venue = V)` | an order for symbol S (default: the symbol of the context) |
| `cancel_all(S)` | the orders of one symbol (`cancel_all(V)`: of one venue) |
| `notional(S)` | `position[S] * market[S].last` (money) |
| `net_exposure`, `gross_exposure`, `beta_exposure` | the sum over all symbols of `position * last`, of `abs(position) * last`, and of `beta * position * last` |
| `history(x, k)` | the value `x` had k events ago (`history(x, 1)` is `prev(x)`; before the run, its initial value); k up to 64 |
| `signals { z: TYPE in [lo, hi]; }` | an input that takes any value of its range at every market update (`int`, `qty`, `price`, `money`, `duration`, `bool`, `real`) |
| `p = any qty in [lo, hi];` | a parameter: any value of the range (also `price`, `money`, `int`, `percent`, …; `any real in [lo, hi]` for reals) |

- **Context.** Where several symbols are declared, a name like `position` needs a symbol: the order's (risk controls, order events, `on new_order` constraints), the event's (`on market_update`), or one given with `[S]`. In an `always` constraint or an assumption, a name without a symbol means **every** symbol (`always position <= 500`: each one). A rule with no symbol in its context (`on clock`) must name the symbol; the verifier says which line.
- **Legs.** `legs { ... }` sends its orders together: the risk controls check each leg with the legs before it counted as open (as a gateway checks one leg after the other), and a rejected leg rejects them all. After that, each leg is an order of its own: it can be acknowledged, filled in part, or rejected by the exchange while the other fills – the legging risk that `examples/19_pairs_legs.gts` finds (a reject of one leg leaves the pair half done, and the next signal opens another pair); `20_pairs_legs_fixed.gts` opens a new pair only from balanced positions and is proved.
- **Signals** make a rule depend on a model you do not want to (or cannot) verify: the verifier proves the constraints for every value the signal can take, at every update. A signal that is fixed for the run is a parameter (`beta = any real in [0.8, 1.2];`).
- **Hedges.** `examples/17_leg_hedger.gts` buys one symbol and hedges each fill with another: starting a new clip while the last clip's hedge is still working grows the unhedged quantity (a 6-event run); `18_..._fixed.gts` waits for the hedges and is proved for any number of orders.
- **Outputs.** Runs name the symbol of every order (`NewOrder 01 BUY 100 INFY @ 28000`), FIX logs carry `Symbol (55)` on every order, execution and market data message, and `--conform` replays logs of several symbols.
- **Exposure proofs.** A risk control that values positions (`gross_exposure`, `notional`) multiplies quantities by prices. For constraints that do not depend on what such a control decides, the verifier also tries a model in which the control may accept or reject any order (its proofs hold for the exact control too); the method then says "(risk controls with products taken as any decision)".

## 19. The strategy pattern library

Most strategies belong to a family, and each family has the constraints it must never break. The library has one verified specification per family, with its standard constraints, to copy and adapt:

```
trade-verify --templates                          # the list
trade-verify --template=pairs > my_pairs.gts      # one template
```

| Template | Family | Standard constraints |
|---|---|---|
| `market_maker` | two-sided quoting with an inventory skew | inventory limits, quotes never cross the market, quote size, limit orders only, kill switch |
| `twap` | a parent order worked evenly over time | never overfilled, on schedule, child size, one side, completes (feasible) |
| `vwap` | a parent order worked along the traded volume | never overfilled, on the curve, volume tracking, child size, one side |
| `pov` | percentage of volume | never overfilled, participation, child size, one side |
| `pairs` | a two-leg spread trade | pair imbalance (legging risk), leg caps, gross exposure checked, beta-weighted exposure |
| `lead_lag` | trade a follower when a leader moves | follower only, position limits, one order at a time, order size |
| `delta_hedge` | keep an options book delta-neutral | every hedge reduces the net delta, no overshoot, hedge limits, hedge size |
| `smart_order_router` | one parent order across venues | no overfill, per-venue exposure, no order after the kill switch, every order cancelling after it |
| `risk_gateway` | pre-trade controls for any strategy | order size, order value, price band, no market orders, position limits, kill switch, accounting |

Every template verifies as it stands (`tests` runs them all). Adapt the parameters and the rules to your strategy, keep the standard constraints, and add your own; a constraint that no longer holds after your change comes back VIOLATED with the run that breaks it. `100 shares` (or `contracts`) is a quantity literal: `position / lot` is then a plain number of lots.

## 20. Verifying your C++ risk checks from their source

Instead of restating your pre-trade checks in GTSL, import them from the C++ code you run. A short binding says what the code means; the specification keeps the assumptions and the constraints.

```
// pretrade.gbind
binding PreTrade;
language cpp;
sources "pretrade_risk.cpp";
flags "-std=c++17";
risk_control PreTradeRisk::allow(o, a) {
    rejects when returns false;
    o.qty      = order.qty;
    o.price    = order.price unit 0.05 market 0;   // the code's price: ticks x 0.05 INR; 0 for a market
     order
    o.isBuy    = order.side == buy;
    o.isMarket = order.is_market;
```

```
    a.position = position;
    a.openBuyQty = open_qty(buy);
    a.killSwitch = kill_switch;
}
```

```
// pretrade.gts
spec PreTradeImported;
units { price = tick(0.05 INR); money = paise; }
ranges { price 20000 .. 40000; order_qty 1 .. 100000; }
mode controls_only;
import "pretrade.gbind";
constraints {
    order_value: always on new_order: notional(order) <= 500000 INR;
    short_limit: always position >= -maxShort;          // maxShort: a member of the C++ class
}
```

- Every `return false` in the check becomes a risk control named after the function and its line; a violated constraint's run shows which C++ line let the order through. `trade-verify pretrade.gts --import-only` prints the generated GTSL.
- Members of the class with an initial value become parameters (override one with `parameter NAME = VALUE;` in the binding, or with `-P NAME=VALUE`).
- `double` arithmetic is verified as binary64, exactly as the compiled code computes it; `unit` converts the verifier's ticks into the code's currency values.
- Code the importer does not support yet (loops, containers, pointers, member calls other than plain getters) stops the import with its file and line – it is never approximated. Calls without effect on trading (logging) are listed with `ignore NAME;`.

`importer/examples/cpp_pretrade` has a risk layer with two bugs found this way (market orders skip the value check; the short check forgets working sells) and its fix.

### Importing the strategy itself

A strategy class – its event handlers and the orders they send – is imported the same way. The binding maps the class's handler methods to events, the calls that send orders to actions, and member reads to what the verifier knows:

```
// mm_fixed.gbind
binding MarketMaker_fixed;
language cpp;
sources "market_maker_fixed.cpp";
strategy MarketMaker {
    on market_update -> onTick(t) { t.bid = market.bid unit 0.05; t.ask = market.ask unit 0.05; }
    on fill -> onTrade(tr) { tr.qty = fill.qty; tr.price = fill.price unit 0.05; tr.isBuy = order.side ==
     buy; }
    on kill_switch -> onKill();
    openBuyQty() = open_qty(buy);
    openSellQty() = open_qty(sell);
    action buy -> sendBuy(qty, limit unit 0.05);
    action sell -> sendSell(qty, limit unit 0.05);
    action cancel_all -> cancelAll();
    range lastMid 0.0 .. 3000.0;
    range inventory -1000 .. 1000;
}
```

- Members the handlers change are the strategy's state; the others are parameters.
- A `double` member needs a `range`; an integer member may have one. Each range is checked (`range: lastMid`), and a proof counts only when its ranges are proven too – a range helps the proof, it is never taken on trust.
- The constraints are written in the specification as usual (`always position <= maxInventory`, `always on new_order where order.side == sell: order.price > market.bid`, `after kill_switch: never new_order`).

`importer/examples/cpp_clip` (a clip buyer: overbuys and keeps two orders working) and `importer/examples/cpp_mm` (a market maker whose skewed quotes can cross the spread and whose inventory check ignores

working orders) each have a buggy and a fixed version: the bugs are found as runs, the fixed versions prove every constraint.

`contract METHOD { requires COND; ensures COND; }` gives a method the handlers call (a routine several callbacks share, like `do_calc`) conditions in GTSL terms that hold on entry and on return wherever it is called. The importer writes them as `assert METHOD_requires: ...;` before and `assert METHOD_ensures: ...;` after every inlined call: each is proven (once, for all calls) and then helps the proofs of the constraints.

**Python.** `language python;` imports a strategy class written in Python with the same binding (handlers, `action buy -> gateway.send_buy(qty, limit unit 0.05);`, mapped reads such as `tick.bid = market.bid unit 0.05;`, `ignore logger.info;`). The source is parsed by Python's own `ast` module (`python3` on the PATH; the code is parsed, never run). `__init__`'s constant members are the parameters and the initial state; the handlers may use assignments, `if/elif/else`, early returns, `for i in range(A, B)` with constant bounds, `min/max/abs/int/ float`, and the class's own methods (inlined; a method used as a value must end in `return EXPR`). Python's `/` is true division and `//`, `%` floor, as in Python. Not yet: setup methods, order-id sets, `round()` (half to even), risk-control functions (C++ only). `importer/examples/py_clip` is the clip buyer in Python (buggy and fixed).

**Java.** `language java;` imports a strategy class written in Java the same way. The source is parsed by javac's own tree API (a JDK's `java` on the PATH; the code is parsed, never compiled or run). The fields of type `int`, `long`, `double` and `boolean` are the parameters and the state (their initialisers, or `this.x = constant;` in the constructor; Java's defaults otherwise); fields of other types (a gateway, a logger) are handles. A bare field name is the field (as in Java), `Math.min/max/abs` and casts are supported, and `/` and `%` are Java's (truncating on integers). `importer/examples/java_clip` is the clip buyer in Java.

Loops: `for (int i = A; i < B; ++i)` (C++) and `for i in range(A, B)` (Python) with bounds that are constants once the parameters are known are unrolled (at most 32 iterations): `importer/examples/cpp_ladder` sends a ladder of orders.

`orders NAME;` names a member set of order ids (`std::set`, `std::unordered_set`, ...): its size becomes `NAME_count` `tracks reported_open_orders`, checked at every step (`tracked:NAME_count`) – the set holds exactly the orders the strategy has been told are working.

`importer/examples/real_wondertrader_hft` imports WonderTrader's HFT demo strategy, unmodified. Its engine delivers a fill as two callbacks (`on_trade` after the position is booked, `on_order` with the order's status), in the broker's order: `hft.gts` (`fill_reports`) finds that the demo can hold two working orders and a position of 2 when a status report and its trade arrive apart (`on_order` calls `do_calc` before the trade is booked, `on_trade` calls it before the order leaves the set); `hft_atomic.gts` (one message per fill) proves every constraint.

## 21. What a result means

- **PROVEN** holds for every run of the model the specification describes: under its assumptions, declared ranges, protocol and options, and with the strategy's rules (or any strategy). It does not cover behaviour the specification does not state – check production with conformance replay.
- **VIOLATED** is reported only for runs of the exact model, replayed concretely; the FIX log of the run replays it.
- **Time** advances in whole time units; events closer together than the unit are ordered arbitrarily (sound, may give false alarms).
- **Symbols** share one currency and one cash account; replace (35=G) is not modelled; multi-leg orders are sent as separate orders that pass the risk controls together (`legs`), not as exchange-native combination orders.
- Long-horizon `feasible` targets (a parent order completed child by child over a full session) may stay UNDECIDED; check them with a smaller quantity.

## 22. The library API

The CLI is one client of the libraries (headers in `include/giolit`):

```
#include "giolit/License.hpp"
#include "giolit/Trade.hpp"
#include "giolit/Report.hpp"

giolit::TokenGrant g = giolit::acquireToken();
```

```
giolit::TradeOptions o;
o.seconds = 60;
giolit::TradeResult r = giolit::verifySpec(g.token, "desk.gts", o);
for (const giolit::ConstraintResult& c : r.constraints)
    std::cout << c.name << " " << giolit::toString(c.verdict) << "\n";   // c.trace: the run; c.proof: the
     invariants
giolit::addHeadroom(g.token, "desk.gts", o, r);                          // c.worst, c.relative, c.stars
giolit::ConformResult cr = giolit::conformLog(g.token, "desk.gts", "dropcopy.fix", o.lower);
```

Link with `-lgiolit_trade -lgiolit_frontend -lgiolit_backend -lgiolit_license -lgiolit_interface` (and `-lgiolit_report` for `writeReport`). `sdk-examples/` has complete programs.

## 23. Command reference

```
trade-verify [options] spec.gts
  --timeout=SECONDS   total time budget (default 60)
  --controls-only     the risk controls against any strategy
  --constraint=NAME   only this constraint
  --slots=K           live orders tracked exactly in the counterexample model (default 4)
  --model=MODE        race (default), slots, aggregate
  -P NAME=VALUE       override a parameter (VALUE `any`: arbitrary)
  --no-sanity         skip the sanity checks
  --json=FILE         results as JSON
  --traces=DIR        every run as .txt, .json, .fix, .csv
  --headroom          risk headroom and the Giolit Risk Resilience Score
  --optimize          the objective's best and worst value
  --report=DIR        the evidence bundle (implies --headroom); --issued-to=NAME
  --conform=LOG       replay a FIX log through the specification
  --verbose           full runs, every sanity check, the replay
  --templates         the strategy pattern library; --template=NAME prints one
  --import-only       the GTSL a specification's `import "x.gbind";` generates from your code
  --activate=FILE     apply a license file; --license; --version; --help
```